1. LE BESOIN

CDA – Travaux pratiques

PROJET 1

On se propose de réaliser un programme qui permet de gérer un stock de produits et les *bons de commande* d'un *site* de vente en ligne.
Le stock contient les produits identifiés par une référence.
Chaque bon de commande contient, les *références* des *produits* commandés.

A terme le programme devra être capable de :

- 1. lire deux fichiers texte (fournis) contenant les produits du stock et les bons de commande.
- 2. afficher tous les produits du stock du site
- 3. afficher tous les bons de commande du site
- 4. afficher un bon de commande en particulier
- 5. réaliser la livraison de tous les bons de commande non livrées : le stock des produits est mis à jour en décrémentant la quantité des produits, les commandes correctes sont marquées comme étant livrées, les commandes incorrectes sont marquées comme non livrées et un texte associé à la commande précise les produits non disponibles et leurs quantités non disponibles.
- 6. modifier les quantités des références de produit d'un bon de commande non livré.
- 7. affiche la somme en euros totale de vente des bons de commande livrés.
- 8. envoyer un état des bons de commande livrés par email.
- 9. envoyer un backup des produits du stock et du bons de commande par FTP sur greta.bretagne-sud.org.

2. ETAPE 1

Dans cette étape on se propose de réaliser les 4 premières fonctionnalités .

Les classes que vous devez créer et/ou modifier sont :

- **Site** Cette classe est la description du site qui contient:
  - la liste des bons de commande (ArrayList<Commande>)
  - la liste des produits en stock (ArrayList<Produit>)

- **Produit** Cette classe est la description d'un produit. Un produit est caractérisé par :
  - la référence du produit (String) (unique) (utilisé dans les bons de commandes)
  - le nom du produit (String)
  - le prix du produit (double)
  - la quantité du produit en stock (int)

- **Commande** Cette classe est la description d'un bon de commande. Un bon de commande est caractérisé par :
  - le numéro du bon de commande (int) (unique)
  - la date de création (String) au format JJ/MM/AAAA
  - le nom du client (String)
  - la liste des références des produits commandés (ArrayList<String>)

Pour démarrer votre programme vous partez du code fourni (projetcda1.zip) dans lequel vous trouvez : une classe **Projet** avec une méthode main qui crée le **Site** et le **GUISite**.
Le GUISite prend en entrée le Site.

Le constructeur de la classe **Site** doit lire les deux fichiers textes :

- le fichier data/Produits.txt

  Chaque ligne de ce fichier est un produit du stock, de la forme :
  ref_produit;nom;prix;quantité

  Exemple :

```
LIVRE-1;Les Miserables de Victor Hugo ;8.50;6
LIVRE-2;La Peste de Albert Camus;8.5;4
VETEMENT-1;Pantalon bleu taille 44 homme;12.0;5
VETEMENT-2;Pantalon jean femme 48;23.0;4
VETEMENT-3;T-shirt rose femme 52;30.0;6
LIVRE-3;Le Père Goriot de Honoré de Balzac;11.0;5
LIVRE-4;L'étranger de Albert Camus;11.0;5
LIVRE-5;Voyage au bout de la nuit de Louis-ferdinand Céline;11.0;3
BD-1;Tintin au Tibet;11.5;2
BD-2;Spirou et Fantasio;11.5;2
```

- le fichier data/Commandes.txt

  Chaque ligne de ce fichier est un produit d'une commande (pas nécessairement dans l'ordre), de la forme :

num_commande;date;client;ref_produit_commandé,ref_produit-commandé
ref_produit_commandé est la référence et la quantité du produit commandé qui est au format : **reference=quantite**
où reference est la référence du stock et quantite la quantité du produit commandé.

Exemple :

```
1;21/09/2022;Macron Brigitte;LIVRE-1=1
1;21/09/2022;Macron Brigitte;LIVRE-2=2
1;21/09/2022;Macron Brigitte;;VETEMENT-1=3
2;22/09/2022;Chirac Bernadette;VETEMENT-2=3
2;22/09/2022;Chirac Bernadette;VETEMENT-3=7
2;22/09/2022;Chirac Bernadette;LIVRE-3=7
3;23/09/2022;Martin Jacques;LIVRE-1=8
4;23/09/2022;Bruni Carla;LIVRE-1=9
```

(Vous utilisez la méthode String[] Terminal.lireFichierTexte(String nomFichier); pour lire les fichier)
(La méthode String[] split(String ligne,4) permet de décoder une ligne du fichier contenant 4 champs).

Pour traiter le fichier des produits du stock :

Pour chaque ligne lue, on crée un produit et on l'ajoute à la liste des produits du stock

Pour traiter le fichier des bons de commande :

Pour chaque ligne lue, on crée le bon de commande que l'on ajoute à la liste des commandes s'il n'existe pas déjà, puis on ajoute la référence du produit (reference=quantite) au bon de commande.

La classe **GUISite** crée un formulaire dans lequel :

- deux boutons permettent d'afficher tous les produits et tous les bons de commande (chaque produit ou commande est affiché sous la forme d'une chaîne de caractère)
- un texte de saisi et un bouton permettant d'afficher la commande d'une commande donné. Vous devez tester si le numéro de commande saisi existe ou pas.

Pour réaliser ces trois actions, codez dans la classe Site les trois méthodes suivantes :

public String listerTousProduits()

Cette méthode retourne sous la forme d'une chaine de caractère, tous les produits (une boucle qui appelle la méthode toString de chaque produit)

public String listerToutesCommandes()

Cette méthode retourne sous la forme d'une chaine de caractère, toutes les commandes (une boucle qui appelle la méthode toString de chaque commande)

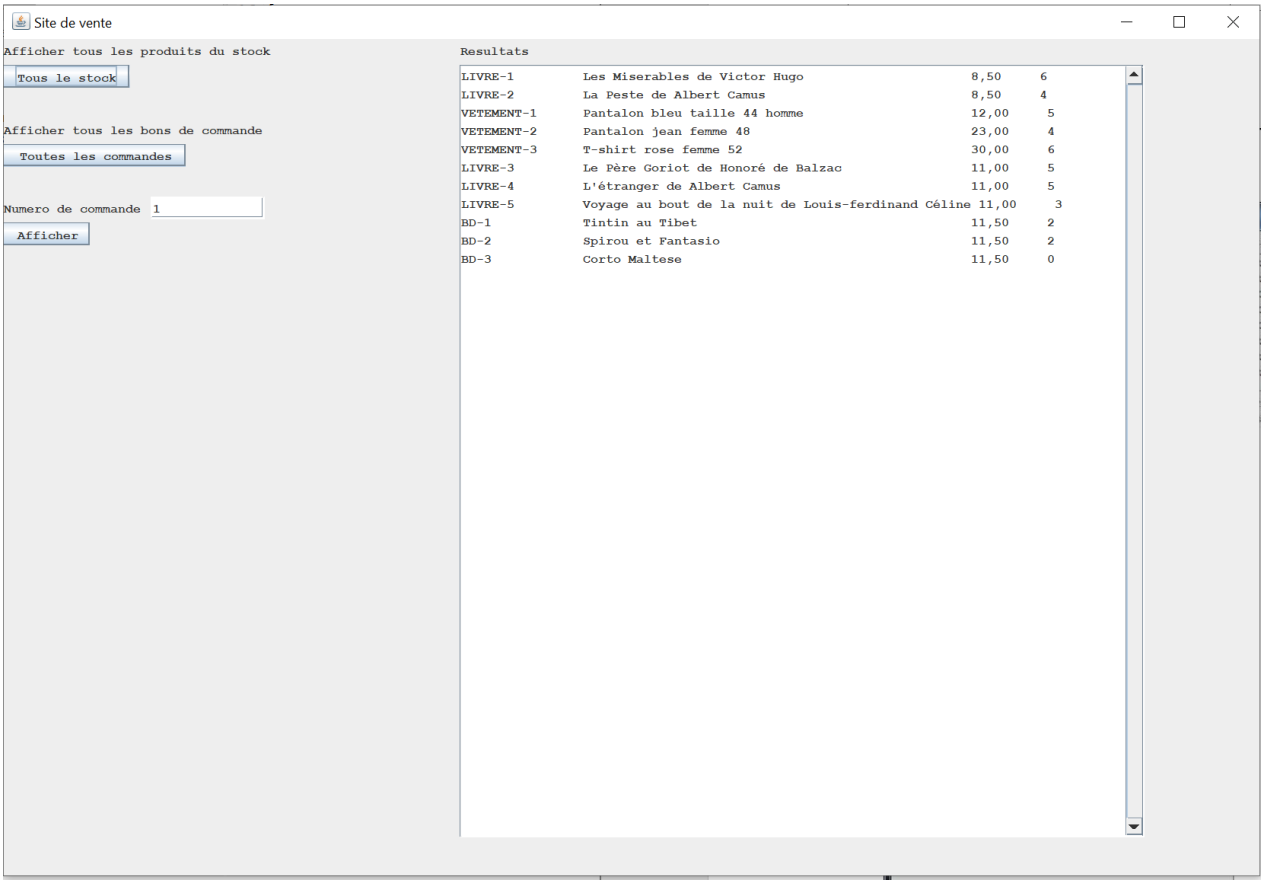
public String listerCommande(int numero)

Cette méthode retourne sous la forme d'une chaine de caractère la commande de numero.

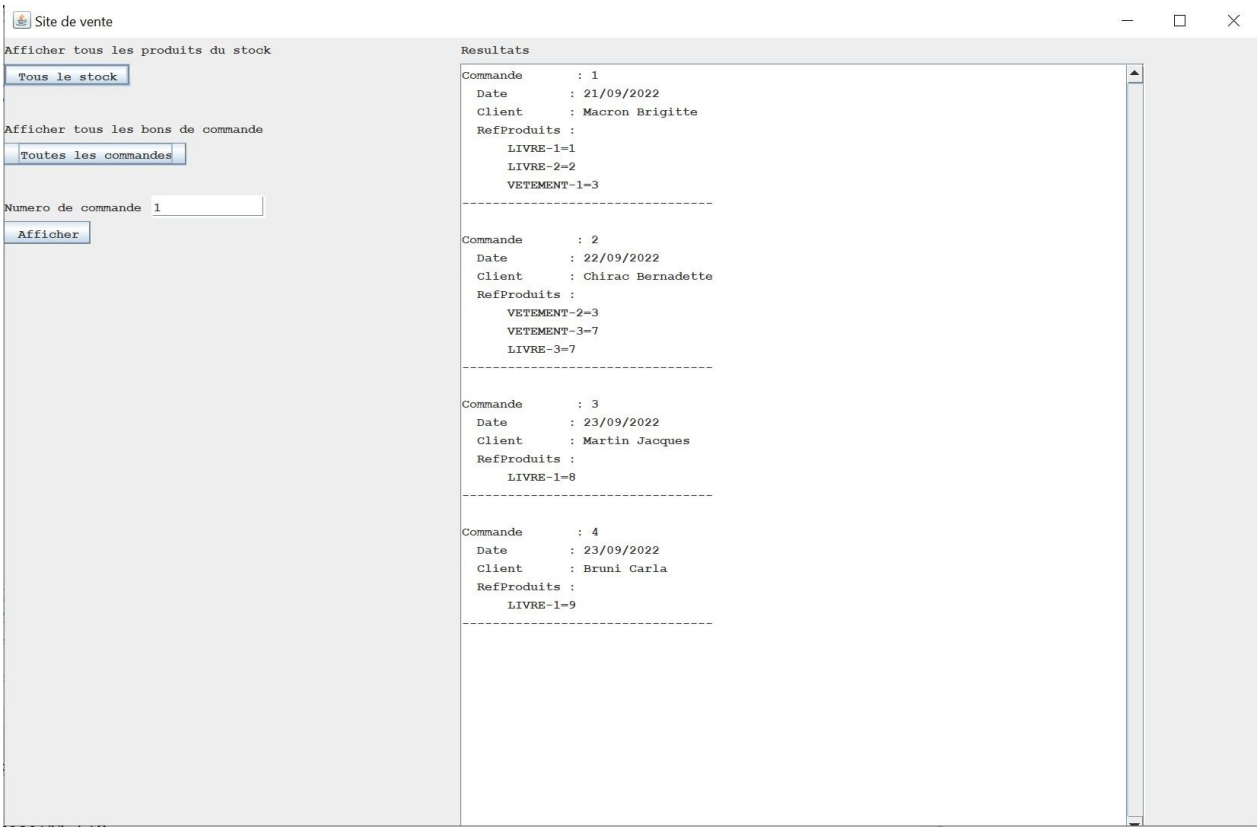
Ces trois méthodes sont appelées dans la méthode **submit** de la classe GUISite.

Nous supposons que les fichiers textes ne contiennent pas d'erreur de syntaxe: pas de "point-virgule" en moins, toutes les informations sont correctement écrites.

La méthode listerTousProduits affiche ce résultat:



La méthode listerToutesCommandes affiche ce résultat :



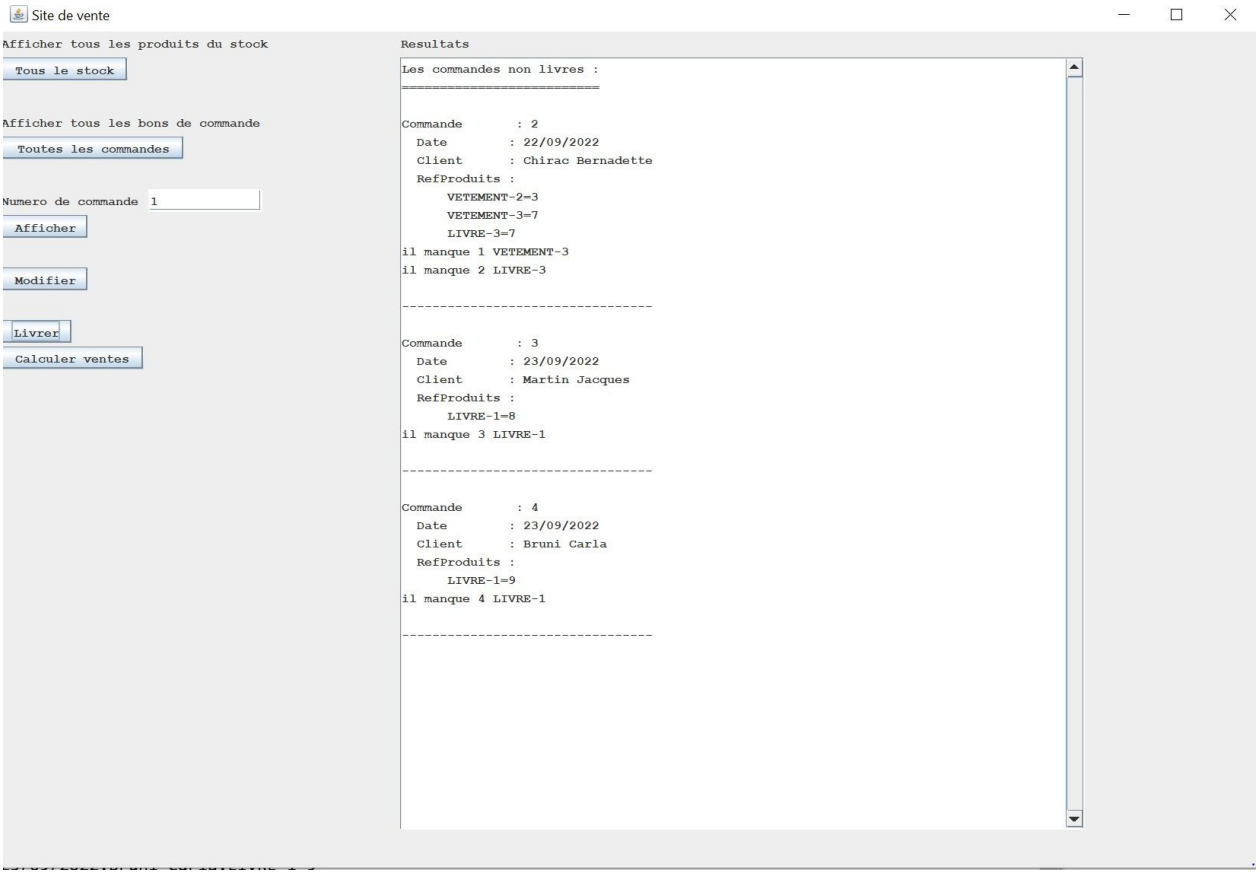
Dans cette étape on se propose de réaliser la fonctionnalité 5.

Vous devez ajouter dans la classe Commande, les deux attributs qui permettent de :

- savoir si le bon de commande a été livré ou pas (boolean). Par défaut, il est à faux.
- un texte libre (String) qui précise les raisons pour lesquelles le bon de commande n'a pas pu être livré.

Vous créez dans la classe Site la méthode qui réalise le traitement demandé, et vous ajoutez dans l'IHM un bouton permettant d'exécuter la méthode. Cette méthode n'a pas de paramètre en entrée. Elle retourne la liste de tous les bons de commandes non livrés.

Exemple d'exécution :



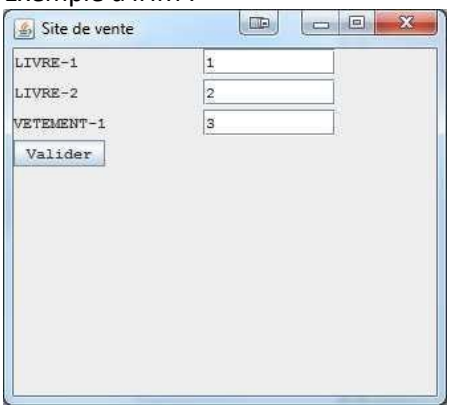
Dans cette étape on se propose de réaliser les fonctionnalités 6 , 7, 8 et 9.

Pour la fonctionnalité 6 , il faut créer une nouvelle IHM dédié à la modification d'un bon de commande non livré.

On saisi un numéro de bon de commande (déjà fait dans l'étape 1) et on clique sur un nouveau bouton de l'IHM principale (exemple "Modifier"). Le rôle de ce bouton est de créer une nouvelle IHM (à l'image de IHMSite) qui est une classe qui s'appelle par exemple **GUIModifierCommande**.

Cette IHM affiche des zones de saisies permettant de modifier les quantités des références de produit du bon de commande. Un bouton "Valider" permet de valider les modifications et fermer la fenêtre.

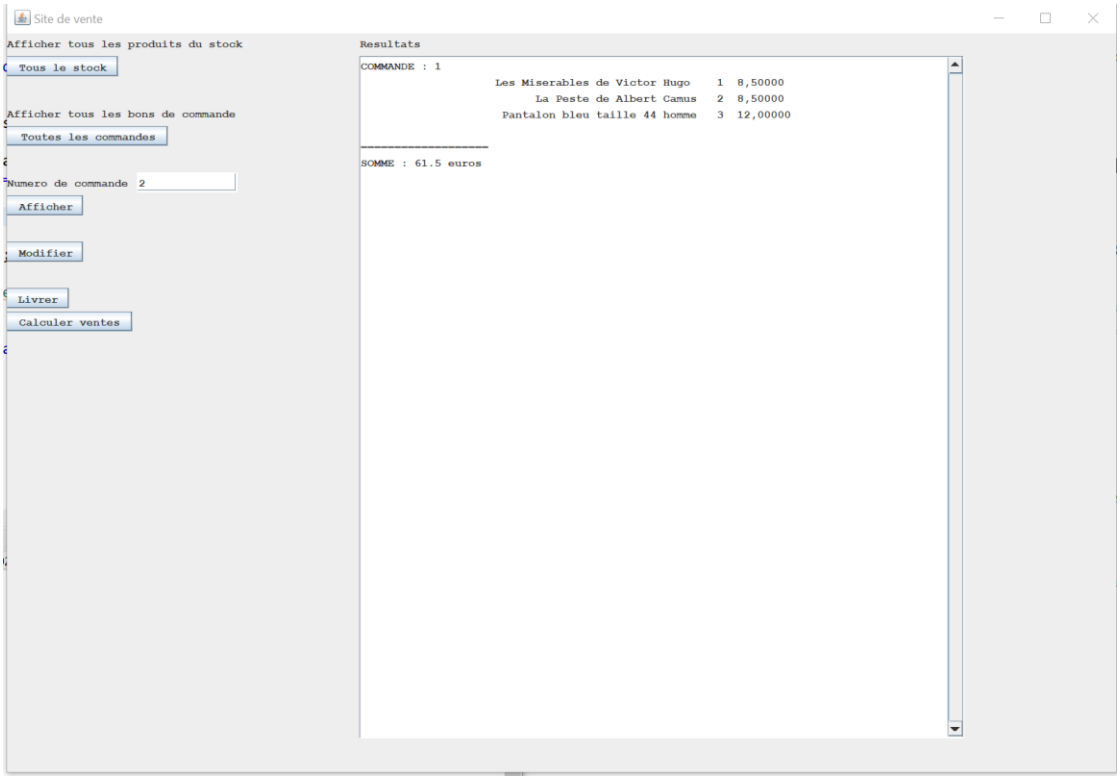
Exemple d'IHM :



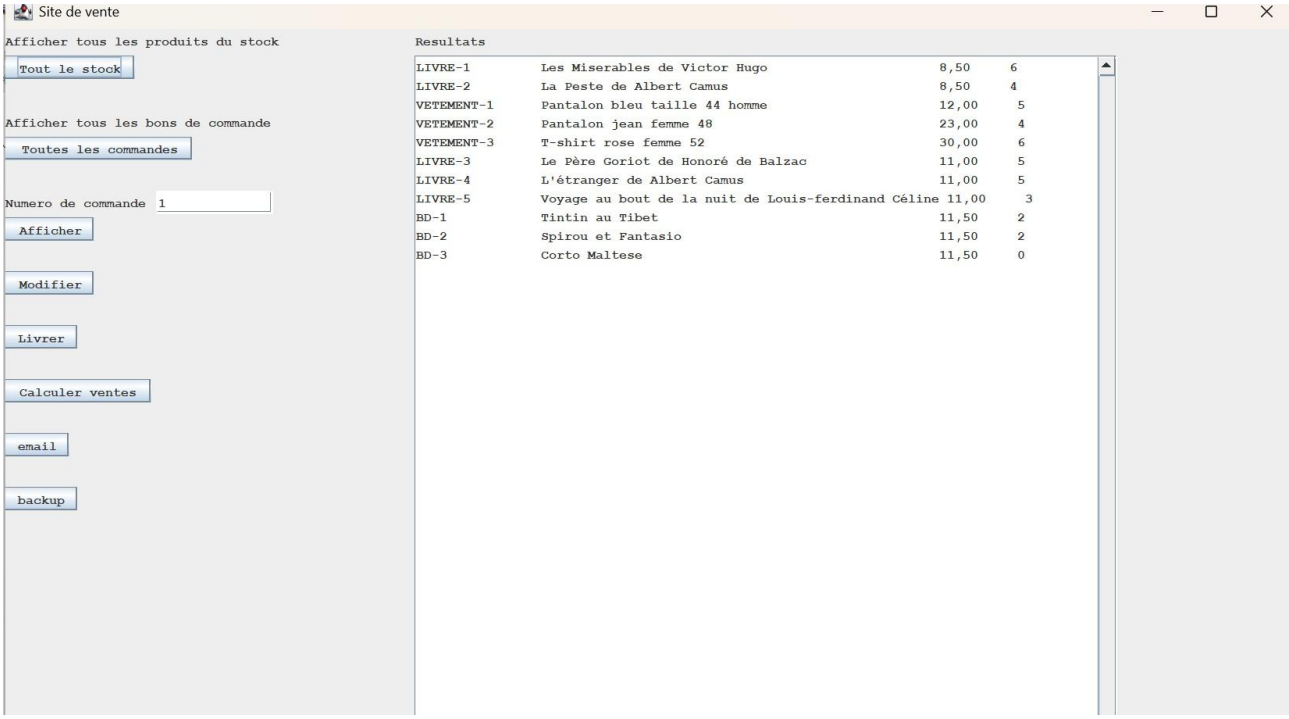
La fenêtre se ferme une fois la validation réalisée.

Si le bon de commande est livré on ne peut pas la modifier.

Pour la fonctionnalité 7 , vous ajoutez un bouton dans l'IHM qui permet de faire le traitement et le résultat est affiché dans la zone de résultat.



Pour la fonctionnalité 8, vous devez ajouter un bouton nommé EMAIL déclenchant l'envoi d'un email contenant un état sous forme de fichier texte correspondant à l'ensemble des bons de commande livrés. Le destinataire, sujet et corps du courriel seront définis en tant que constante. Une fenêtre modale apparaitra indiquant la réussite ou l'echec du traitement.



Pour la fonctionnalité 9 , vous devez créer le bouton backup. Le clic sur ce bouton déclenche la sauvegarde des fichiers de données en FTP sur le serveur greta-bretagne-sud.org. Vous prendrez soin de chiffrer les fichiers avec l'algorithme AES .Une fenêtre modale apparaitra indiquant la réussite ou l'echec du traitement.

5. QUELQUES PRECISIONS

Les règles de codage que vous devez suivre sont :

- sauf pour les constantes, les attributs de toutes les classes doivent être privés
- toutes les classes commencent par une majuscule
- tous les attributs, méthodes, paramètres et variables commencent par une minuscule
- les noms de tous les répertoires de package sont en minuscule
- les packages créés commencent tous par "fr.cda"
- une indentation stricte est respectée dans tout le code
- un minimum de commentaire est nécessaire afin de comprendre ce que fait le code. Surtout quand ce que fait le code n'est pas décrit précisément dans le sujet.
- Une JAVADOC doit être réalisée.

Conseils :

- Ecrivez peu de code à chaque fois afin d'avoir toujours un code qui se compile correctement
- Après chaque étape ou sous-étape, le programme s'exécutant correctement, faites une sauvegarde des sources. Cela permet de pouvoir revenir en arrière si besoin et permet aussi de pouvoir donner à l'enseignant une version du programme qui s'exécute correctement et qui remplit correctement une partie des fonctionnalités demandées dans le sujet.
- Dans un premier temps ne faites pas plus que le sujet ne le demande. Une fois que le programme respecte le sujet, vous faites une sauvegarde et vous pouvez apporter des améliorations qui sont toujours appréciées par le correcteur, à condition qu'elles soient bien commentées.

6. ARCHITECTURE DES REPERTOIRES

L'architecture suivante est une proposition de création des répertoires et des fichiers pour réaliser votre projet. Vous pouvez adapter cette architecture à votre convenance.

projetCDA1
- **data**
  - Produits.txt
  - Commandes.txt
- **src**
  - **fr**
    - **cda**
      - **projet**
        - Projet.java (*main*)
        - Site.java
        - Produit.java
        - Commande.java
        - GUISite.java
        - GUIModifierCommande.java
      - **ihm**
        - Formulaire.java
        - FormulaireException.java
        - FormulaireInt.java
        - ....
      - **util**
        - DateString.java
        - Terminal.java
        - TerminalException.java
        - ...
- **bin**
  - <les .class>

Les classes en rouge sont des classes qui vous sont données.

Le répertoire **data** contient les fichiers de données.

7. soutenance

Le projet sera présenté le 8 Novembre.

-Le projet sera évalué sur la base du code source et d'un mini rapport de quelques pages.

Le rapport est composé de deux parties :

- Guide utilisateur, décrivant l'application, ses principales fonctionnalités et comment l'utiliser.
- Un guide développeur également qui présentera l'architecture logicielle choisie, les classes et interfaces utilisées,le modèle de données manipulées par l'application et une description des algorithmes utilisés.

Vous devez déposer votre projet au format zip et votre rapport sur Moodle.

Vous devrez présenter ce projet d'un point de vue fonctionnel et technique.

La durée de la soutenance est de 25 minutes.
.La réalisation de deux modifications (une simple et une plus complexe) de code devra être réalisée et expliquée au moment de la soutenance.